

CSS3 Tutorial: Intermediate Level

For people comfortable with the basics — box model, Flexbox, Grid, and media queries — who want to write cleaner, more powerful CSS.

Table of Contents

1. [CSS Custom Properties \(Variables\)](#)
 2. [The Cascade and Specificity, Properly](#)
 3. [Advanced Selectors](#)
 4. [Pseudo-Elements for Content Tricks](#)
 5. [CSS Functions: calc, min, max, clamp](#)
 6. [Grid Template Areas](#)
 7. [Flexbox Deep Dive: Wrapping and Shrinking](#)
 8. [Transforms](#)
 9. [Animation Timing and Multi-Step Keyframes](#)
 10. [Stacking Context and z-index](#)
 11. [Container Queries](#)
 12. [Naming Conventions: BEM](#)
 13. [Practice Project: A Responsive Dashboard Card](#)
 14. [Where to Go Next](#)
-

1. CSS Custom Properties (Variables)

Custom properties let you define a value once and reuse it everywhere, and update it dynamically (even with JavaScript).

```
:root {
  --brand-color: #2563eb;
  --spacing-unit: 8px;
  --radius: 6px;
}

.button {
  background: var(--brand-color);
  padding: calc(var(--spacing-unit) * 2);
  border-radius: var(--radius);
}
```

`:root` targets the top of the document, so these variables are available everywhere. You can also scope them to a component:

```
.card {
  --card-padding: 16px;
  padding: var(--card-padding);
}
```

Variables can be overridden per context — this is how a lot of dark-mode toggles work:

```
:root { --bg: white; --text: black; }
[data-theme="dark"] { --bg: #111; --text: #eee; }
}
```

```
body { background: var(--bg); color: var(--text); }
```

2. The Cascade and Specificity, Properly

You've likely hit the "why isn't my CSS applying" problem. Specificity is calculated as a score with three tiers:

Selector type	Weight
Inline style	Always wins over any selector
ID (<code>#header</code>)	1-0-0
Class, attribute, pseudo-class (<code>.card</code> , <code>[type="text"]</code> , <code>:hover</code>)	0-1-0
Element, pseudo-element (<code>div</code> , <code>::before</code>)	0-0-1

Higher tiers always beat lower ones, regardless of count: one ID beats a hundred classes. Within the same tier, the rule that appears later in the file wins.

`!important` overrides all of this — use it sparingly, mainly as an escape hatch, since it makes future debugging harder.

3. Advanced Selectors

Beyond the basics, these let you avoid adding extra classes to your HTML:

```
/* Attribute selectors */
input[type="email"] { }
a[href^="https://"] { } /* starts with */
a[href$=".pdf"] { }      /* ends with */
a[href*="example"] { }   /* contains */

/* Negation */
li:not(.active) { }

/* Structural */
li:nth-child(3n + 1) { } /* 1st, 4th, 7th...
formula-based selection */
p:only-child { }
tr:nth-of-type(even) { }

/* Combinators */
h2 + p { } /* the <p> immediately after an
<h2> */
h2 ~ p { } /* every <p> that is a sibling
AFTER an <h2> */
```

`:nth-child(A $n$  + B)` is worth practicing: `A` is the step size, `B` is the offset. `2 $n$` = even, `2 $n$ +1` = odd, `3 $n$` = every third.

4. Pseudo-Elements for Content Tricks

`::before` and `::after` insert generated content without touching your HTML:

```

.required::after {
  content: " *";
  color: red;
}

.quote::before {
  content: "\201C"; /* opening curly quote */
}

.tooltip::before {
  content: attr(data-tooltip); /* pull from a
data attribute */
}

```

These require `content:` to render at all, even if it's just an empty string — useful for pure decorative shapes:

```

.card::before {
  content: "";
  position: absolute;
  width: 4px;
  background: var(--brand-color);
}

```

5. CSS Functions

```

/* calc() mixes units */
width: calc(100% - 40px);

/* clamp(min, preferred, max) – responsive
sizing with no media query */
font-size: clamp(1rem, 2vw + 1rem, 2.5rem);

```

```
/* min() and max() */
width: min(90%, 600px); /* whichever is
smaller */
height: max(300px, 50vh); /* whichever is larger
*/
```

`clamp()` is the standout here: it replaces most "font-size gets bigger on desktop" media queries with a single line that scales fluidly between a floor and a ceiling.

6. Grid Template Areas

Named grid areas make complex layouts genuinely readable in the CSS itself:

```
.layout {
  display: grid;
  grid-template-columns: 200px 1fr;
  grid-template-rows: 60px 1fr 40px;
  grid-template-areas:
    "sidebar header"
    "sidebar main"
    "sidebar footer";
  min-height: 100vh;
}
.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main { grid-area: main; }
.footer { grid-area: footer; }
```

You can literally see the shape of the page laid out in the `grid-template-areas` string — a big readability win over

tracking column/row numbers.

7. Flexbox Deep Dive

Two properties that solve most "why won't this wrap/shrink correctly" frustrations:

```
.container {  
  display: flex;  
  flex-wrap: wrap; /* items drop to a new line  
instead of overflowing/squishing */  
}  
  
.item {  
  flex: 1 1 200px;  
  /* flex-grow: 1    -> can grow to fill space  
*/  
  /* flex-shrink: 1 -> can shrink if space is  
tight    */  
  /* flex-basis: 200px -> starting size before  
growing/shrinking */  
}
```

This single-line pattern (`flex: 1 1 200px;` on cards, `flex-wrap: wrap;` on the container) is how most "responsive card grid" layouts are built without a single media query.

8. Transforms

Transforms move, rotate, scale, or skew an element visually without affecting the layout of surrounding elements:

```
.card:hover {  
  transform: translateY(-4px) scale(1.02);  
}  
  
.icon {  
  transform: rotate(45deg);  
}
```

Multiple transforms can be chained in one declaration, applied in the order written. Because transforms don't trigger layout recalculation, they're also the most performant way to animate movement (prefer `transform` over animating `top / left`).

9. Animation Timing

Beyond a simple two-keyframe fade, you can define multiple stops:

```
@keyframes pulse {  
  0%   { transform: scale(1); opacity: 1; }  
  50%  { transform: scale(1.1); opacity: 0.7; }  
  100% { transform: scale(1); opacity: 1; }  
}  
  
.badge {  
  animation: pulse 2s ease-in-out infinite;  
}
```


Useful animation properties:

```
animation-delay: 0.3s;  
animation-iteration-count: infinite; /* or a  
number */  
animation-fill-mode: forwards; /* keep the last  
keyframe's styles after it ends */
```

10. Stacking Context and z-index

`z-index` only works on positioned elements (`relative` , `absolute` , `fixed` , `sticky`), and it's not global — each positioned ancestor with a set `z-index` creates its own **stacking context**, meaning `z-index` values only compete against siblings within the same context.

```
.modal {  
  position: fixed;  
  z-index: 1000;  
}
```

If a high `z-index` still isn't appearing on top, the likely cause is a parent element with its own stacking context (often from `position` , `opacity < 1` , or `transform`) trapping it underneath.

11. Container Queries

Where media queries respond to the *viewport*, container queries respond to the size of a *parent container* — letting the same component adapt differently depending on where it's placed on the page.

```
.card-container {  
  container-type: inline-size;  
  container-name: card;  
}  
  
@container card (min-width: 400px) {  
  .card {  
    flex-direction: row;  
  }  
}
```

This is newer than media queries but well-supported in current browsers, and is a genuine shift toward components that are responsive to their own context rather than the whole page.

12. Naming Conventions: BEM

As stylesheets grow, naming collisions and unpredictable overrides become the biggest source of pain. **BEM** (Block, Element, Modifier) is a widely used convention:

```
.card { }           /* Block */  
.card__title { }    /* Element: a part of  
the block */  
.card__title--large { } /* Modifier: a variation  
*/
```

```
<div class="card card--featured">
  <h2 class="card__title card__title--
large">Title</h2>
</div>
```

The payoff: every class is a single, flat, specificity-0-1-0 selector, so you rarely fight the cascade, and it's immediately clear from the HTML which block an element belongs to.

13. Practice Project

Build a small responsive dashboard using only what's above:

1. Use **Grid Template Areas** for the overall page shell (header, sidebar, main).
 2. Inside the main area, lay out a row of stat cards with `flex-wrap: wrap` and `flex: 1 1 200px` so they reflow on narrow screens.
 3. Define your color palette as **custom properties** in `:root`, and add a `data-theme="dark"` override.
 4. Give each card a hover `transform: translateY(-4px)` with a smooth `transition`.
 5. Use `clamp()` for the dashboard's main heading font size instead of a media query.
-

14. Where to Go Next

- **CSS Grid** `subgrid` — align nested grids to their parent's tracks
- `:has()` — a parent selector; style an element based on what it contains
- **Logical properties** (`margin-inline` , `padding-block`)
— layout that adapts automatically to text direction (LTR/RTL)
- **CSS Nesting** — native nesting (similar to Sass) is now supported in modern browsers
- **A CSS methodology at scale** — look into BEM plus a utility layer, or a framework like Tailwind, once you're maintaining a larger project

Tip: the biggest jump from beginner to intermediate CSS is usually the shift from "fighting layout" to "designing systems" — custom properties and BEM are what make that shift possible.